



Guide multiplateforme apprenant - Sprint 3

Version locale du 24 août 2026. Ce guide complète le pas à pas apprenant sans modifier les objectifs, les preuves ni les résultats attendus. Choisissez une colonne au début du Sprint et gardez le même terminal pendant une séquence.

Le formateur annonce en direct le rythme, les pauses et les transitions. Les commandes et résultats attendus restent identiques sur les trois systèmes.

Poste	Terminal recommandé dans VS Code	Type de commandes
Windows	Windows PowerShell 5.1 ou PowerShell 7	blocs powershell
macOS	zsh, terminal par défaut	blocs bash
Linux	bash	blocs bash

Sous macOS, le terminal interactif reste zsh. Lorsqu'une commande commence par `bash scripts/...`, c'est volontaire : le script est exécuté par bash. Sous WSL, suivez la colonne Linux et gardez le dépôt dans votre dossier Linux, par exemple `~/CISIA`, plutôt que dans `/mnt/c/...`

1. Ouvrir le bon terminal et le bon dossier

Dans VS Code, choisissez Fichier > Ouvrir le dossier, sélectionnez le dépôt ou le TP demandé, puis Terminal > Nouveau terminal.

Windows - PowerShell

```
Get-Location
Test-Path -LiteralPath .\pyproject.toml
Test-Path -LiteralPath .\uv.lock
$PSVersionTable.PSVersion
```

macOS - zsh ou Linux - bash

```
pwd
test -f ./pyproject.toml && echo "pyproject.toml: OK"
test -f ./uv.lock && echo "uv.lock: OK"
printf 'shell=%s\n' "$SHELL"
```

Si un fichier attendu est absent, n'installez rien et ne créez pas un nouveau projet : rouvrez le bon dossier dans VS Code.

2. Préflight commun

Requis dès J1, avec les mêmes commandes sur les trois systèmes :

```
git --version
uv --version
uv sync --frozen --extra dev
uv run python --version
uv run pytest -q
uv run ruff check .
```

Docker n'est requis qu'avant J3. A ce moment, ajoutez :

```
docker --version
docker compose version
```

La version qui fait foi est `uv run python --version`, attendue en Python 3.13.x.

Avec `uv run`, il n'est pas nécessaire d'activer manuellement `.venv`.

Activation facultative, uniquement si le formateur la demande :



```
# Windows PowerShell
.\.env\Scripts\Activate.ps1
```

```
# macOS zsh et Linux bash
source .env/bin/activate
```

Si `uv` manque avant la formation :

```
# Windows avec WinGet
winget install --id=astral-sh.uv -e
```

```
# macOS avec Homebrew
brew install uv
```

```
# macOS ou Linux, installateur officiel Astral
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Fermez puis rouvrez le terminal après installation. Pendant une séquence de cours, ne lancez pas un installateur ou une commande `sudo` improvisée : signalez le blocage et utilisez le plan B du formateur.

Pour Docker, Windows et macOS utilisent normalement Docker Desktop. Linux peut utiliser Docker Desktop ou Docker Engine avec le plugin Compose. Dans tous les cas, la commande attendue est `docker compose`, jamais l'ancien `docker-compose`.

Le parcours officiel n'utilise plus le correctif historique du dépôt précédent.

Au signal du formateur, chargez seulement le numéro court annoncé, après avoir commité votre travail. Le script crée une branche locale de sauvegarde avant la fusion ; il ne supprime ni ne réécrit vos commits.

```
# Windows PowerShell - exemple jalon 03
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 03
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 03
```

```
# macOS zsh ou Linux bash - même jalon
bash scripts/formation/mettre_a_niveau.sh 03
bash scripts/formation/verifier_jalon.sh 03
```

N'appliquez jamais un ancien fichier `.patch` au dépôt Parcours. En cas de conflit, laissez le script annuler la fusion et appelez le formateur ; le mode de rattrapage documenté préserve votre branche et sa sauvegarde.

3. Traductions indispensables

Carte Windows - PowerShell

- Dossier et fichiers : `Get-Location`, puis `Get-ChildItem`.
- Tester puis lire un fichier : `Test-Path -LiteralPath .\fichier`, puis `Get-Content -LiteralPath .\fichier`.

Copier `.env` seulement s'il manque :

```
if (-not (Test-Path -LiteralPath .\env)) {
    Copy-Item -LiteralPath .\env.example -Destination .\env
}
```

- Variable temporaire : `$env:NOM = "valeur"`.
- GET HTTP en échec explicite :
`Invoke-RestMethod http://127.0.0.1:8000/health`.
- Chercher du texte :
`Select-String -Path .\fichier -Pattern 'mot'`.
- Dossier temporaire : `$env:TEMP`.



Carte macOS - zsh

- Dossier et fichiers : `pwd`, puis `ls -la`.
- Tester puis lire un fichier : `test -f ./fichier`, puis

`cat ./fichier`.

Copier `.env` seulement s'il manque :

```
if [ ! -f .env ]; then
  cp .env.example .env
fi
```

- Variable temporaire : `export NOM='valeur'`.
- GET HTTP en échec explicite :

`curl -fsS http://127.0.0.1:8000/health`.

- Chercher du texte : `grep -n 'mot' ./fichier`.
- Dossier temporaire : `${TMPDIR:-/tmp}`.

Carte Linux - bash

- Dossier et fichiers : `pwd`, puis `ls -la`.
- Tester puis lire un fichier : `test -f ./fichier`, puis

`cat ./fichier`.

Copier `.env` seulement s'il manque :

```
if [ ! -f .env ]; then
  cp .env.example .env
fi
```

- Variable temporaire : `export NOM='valeur'`.
- GET HTTP en échec explicite :

`curl -fsS http://127.0.0.1:8000/health`.

- Chercher du texte : `grep -n 'mot' ./fichier`.
- Dossier temporaire : `${TMPDIR:-/tmp}`.

Les chemins écrits avec `/`, par exemple `scripts/train_model.py`, fonctionnent avec Python, Git et Docker sur les trois systèmes. Les chemins `C:\...`, les cmdlets `Get-Content`, `Copy-Item`, `Test-Path` et `Invoke-RestMethod` sont propres à PowerShell.

4. Git et jalons de demi-journée

Les commandes Git de base sont communes :

```
git clone URL_ANNONCEE_PAR_LE_FORMATEUR
cd NOM_DU_DEPOT
git switch -c prenom-nom
git status
git add -A
git commit -m "travail avant nouveau jalon"
```

Ne travaillez jamais directement sur `main` ni sur une branche `jalons/...`.

Avant chaque mise à niveau, le dépôt doit être propre.

Le formateur annonce un numéro court de 01 à 12 : utilisez exactement ce numéro. Les anciens slugs longs restent compatibles, mais ne sont plus la notation officielle de la session.

Windows

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 03
powershell -ExecutionPolicy Bypass -File .\scripts\formation\verifier_jalon.ps1 -Jalon 03
```



En cas de conflit :

```
powershell -ExecutionPolicy Bypass -File .\scripts\formation\mettre_a_niveau.ps1 -Jalon 03 -Rattrapage
```

macOS ou Linux

```
bash scripts/formation/mettre_a_niveau.sh 03
bash scripts/formation/verifier_jalon.sh 03
```

En cas de conflit :

```
bash scripts/formation/mettre_a_niveau.sh 03 --rattrapage
```

Les deux variantes créent une branche `sauvegarde/...` avant la fusion. Le mode `rattrapage` annule une fusion conflictuelle et crée une branche `rattrapage/...`; il ne supprime et ne réécrit aucun commit.

5. M23 - package, tests et qualité

Ces commandes sont communes :

```
uv run pytest tests/test_package.py tests/test_loaders.py tests/test_temporal.py -q
uv run ruff check .
uv run indusense --help
```

Ouvrez les fichiers depuis l'Explorateur VS Code. Les chemins affichés avec `/` restent valides sous Windows.

Pour la démonstration Gitleaks sur macOS/Linux :

```
cat > fuite_demo.txt <<'EOF'
# Valeurs d'exemple publiques et invalides.
AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
EOF
git add -f -- fuite_demo.txt
uv run pre-commit run gitleaks --files fuite_demo.txt
git restore --staged -- fuite_demo.txt
rm -- fuite_demo.txt
```

6. M24 - pre-commit, DVC et MLflow

Commandes communes :

```
uv sync --frozen --extra dev --extra mlops
uv run pre-commit run --all-files
uv run pytest -q
git diff --exit-code -- uv.lock
```

Remote DVC local de démonstration :

```
# Windows
$dvcRemote = Join-Path $env:TEMP 'cisla-dvc-store'
uv run python scripts/demo_versioning.py --remote "$dvcRemote"
```

```
# macOS et Linux
dvc_remote="${TMPDIR:-/tmp}/cisla-dvc-store"
uv run python scripts/demo_versioning.py --remote "$dvc_remote"
```

N'ajoutez jamais une vraie clé, un jeton ou un mot de passe dans Git, DVC ou une capture d'écran.

7. M25 - API FastAPI

Ouvrez deux terminaux VS Code. Dans le terminal 1, commande commune :

```
uv run uvicorn indusense.api.main:app --reload --port 8000
```



Dans le terminal 2 :

```
# Windows
Invoke-RestMethod http://127.0.0.1:8000/health
Start-Process http://127.0.0.1:8000/docs
```

```
# macOS
curl -fsS http://127.0.0.1:8000/health
open http://127.0.0.1:8000/docs
```

```
# Linux
curl -fsS http://127.0.0.1:8000/health
xdg-open http://127.0.0.1:8000/docs >/dev/null 2>&1 &
```

Si l'ouverture automatique du navigateur échoue, copiez simplement l'URL dans Chrome ou Firefox. Arrêtez Uvicorn avec **Ctrl+C** dans le terminal 1.

Pour appliquer le paquet de preuves M25 sur les trois systèmes, synchronisez d'abord sans muter le lock, puis lancez l'apporteur Python commun :

```
uv sync --frozen --extra dev
uv run --frozen python FORMATION/EXERCICES/tp_api_m25_v1_20260823/APPLIQUER_PREUVES_M25.py .
```

Si le dossier M25 a été remis à côté du clone, adaptez uniquement son chemin :

```
uv sync --frozen --extra dev
uv run --frozen python ../tp_api_m25_v1_20260823/APPLIQUER_PREUVES_M25.py .
```

8. M26 - sécurité

La suite de tests est commune :

```
uv run pytest tests/test_api.py tests/test_security.py -q
```

Pour afficher la signature sans ligne trop longue, utilisez le bloc de votre système.

Windows PowerShell :

```
$probe = '@'
from inspect import signature
from indusense.api.security import rate_limit_dependency

print(signature(rate_limit_dependency))
'@
$probe | uv run python -
```

macOS zsh ou Linux bash :

```
uv run python - <<'PY'
from inspect import signature
from indusense.api.security import rate_limit_dependency

print(signature(rate_limit_dependency))
PY
```

Les statuts 400, 401, 413, 422 et 429 doivent être produits par les mêmes tests sur les trois systèmes. Ne mettez jamais la clé API dans le code ou le journal Git.

9. M27 et M28 - Docker et Compose

Créer `.env` localement :

```
# Windows
if (-not (Test-Path -LiteralPath .\.env)) {
    Copy-Item -LiteralPath .\.env.example -Destination .\.env
}
```



```
# macOS et Linux
test -f .env || cp .env.example .env
```

Les commandes Docker sont communes :

```
docker run --rm hello-world
docker build -t indusense:0.1.0 .
docker run --rm -d --name indusense -p 8000:8000 --env-file .env indusense:0.1.0
docker inspect indusense --format '{{.Config.User}}'
docker stop indusense
docker compose config -q
docker compose up -d --build
docker compose ps
docker compose down
```

Tester l'API :

```
# Windows
Invoke-RestMethod http://127.0.0.1:8000/health
```

```
# macOS et Linux
curl -fsS http://127.0.0.1:8000/health
```

Sur macOS Apple Silicon, n'ajoutez pas spontanément `--platform linux/amd64` :
utilisez d'abord les images multi-architectures prévues. Sous Linux, si Docker
répond `permission denied` sur `/var/run/docker.sock`, ne relancez pas tout avec
`sudo`; arrêtez-vous et demandez la validation du formateur.

10. M29 et M30 - Prefect et idempotence

Commandes communes :

```
uv run python flows/pipeline.py
uv run python scripts/demo_prefect_idempotence.py
git status --short
```

Si une base SQLite de preuve temporaire est demandée :

```
# Windows
$dbPath = Join-Path $env:TEMP 'indusense-preuve.db'
```

```
# macOS et Linux
db_path="${TMPDIR:-/tmp}/indusense-preuve.db"
```

Ne réutilisez pas une base existante comme preuve d'idempotence.

Exécution complète depuis l'hôte Windows :

```
$env:PREFECT_PROFILE = 'ephemeral'
$env:PREFECT_SERVER_ANALYTICS_ENABLED = 'false'
$env:PREFECT_CLOUD_ENABLE_ORCHESTRATION_TELEMETRY = 'false'
$env:INDUSENSE_DATA_DIR = 'C:\CHEMIN\VERS\DONNEES_COMPLETES'
$sourceData = (Resolve-Path -LiteralPath $env:INDUSENSE_DATA_DIR).Path
$env:COMPOSE_PROJECT_NAME = 'cisia_m30_' + (Get-Date -Format 'yyyyMMdd_HHmss')
docker compose up -d --wait db
docker compose run --rm --no-deps `
  -e PREFECT_PROFILE=ephemeral `
  -e INDUSENSE_DATA_DIR=/app/data/run `
  --volume "${sourceData}:/app/data/run:ro" `
  api python -m indusense.flows.predict_flow
$qaDir = Join-Path $env:TEMP ('cisia_m30_' + (Get-Date -Format 'yyyyMMdd_HHmss'))
New-Item -ItemType Directory -Path $qaDir | Out-Null
$dbPath = Join-Path $qaDir 'predictions.db'
$env:INDUSENSE_DB_URL = 'sqlite:/// ' + ($dbPath -replace '\\', '/')
```

Exécution équivalente sous macOS/Linux :



```
set -e || pipefail
export PREFECT_PROFILE=ephemeral
export PREFECT_SERVER_ANALYTICS_ENABLED=false
export PREFECT_CLOUD_ENABLE_ORCHESTRATION_TELEMETRY=false
export INDUSENSE_DATA_DIR="$HOME/CISIA/donnees-completes"
source_data="$(cd "$INDUSENSE_DATA_DIR" && pwd -P)"
export COMPOSE_PROJECT_NAME="cisla_m30_$(date +%Y%m%d_%H%M%S)"
docker compose up -d --wait db
docker compose run --rm --no-deps \
  -e PREFECT_PROFILE=ephemeral \
  -e INDUSENSE_DATA_DIR=/app/data/run \
  --volume "$source_data:/app/data/run:ro" \
  api python -m indusense.flows.predict_flow
qa_dir="$(mktemp -d "${TMPDIR:-/tmp}/cisla_m30.XXXXXX")"
export INDUSENSE_DB_URL="sqlite:///${qa_dir}/predictions.db"
```

Sur macOS, Docker Desktop doit autoriser le partage du dossier source. Sous un Linux avec SELinux enforcing, le formateur peut valider un montage `:ro,z` ; ne changez pas ce suffixe au hasard.

11. M31 et M32 - PayGuard

Vérifier l'archive avant extraction :

```
# Windows
Get-FileHash -Algorithm SHA256 .\tp_payguard_apprenants.zip
Expand-Archive -LiteralPath .\tp_payguard_apprenants.zip -DestinationPath .\tp_payguard
```

```
# macOS
shasum -a 256 ./tp_payguard_apprenants.zip
ditto -x -k ./tp_payguard_apprenants.zip ./tp_payguard
```

```
# Linux
sha256sum ./tp_payguard_apprenants.zip
unzip ./tp_payguard_apprenants.zip -d ./tp_payguard
```

Ouvrez ensuite le dossier qui contient son propre `pyproject.toml` et `uv.lock`, puis utilisez les commandes `uv` du TP. Ne mélangez pas son environnement avec celui d'InduSense.

12. M31 et M32 - drift InduSense

Utilisez un chemin local court :

Système	Exemple
Windows	C:\CISIA\S3\tp_drift_indusense
macOS	~/CISIA/S3/tp_drift_indusense
Linux	~/CISIA/S3/tp_drift_indusense

Préflight commun :

```
uv sync --frozen --extra dev
uv run python --version
uv run python -m pytest tests -q -p no:cacheprovider
```

Les commandes Python sont communes si les chemins utilisent / :

```
uv run python scripts/train_model.py
uv run python scripts/drift_lab.py --fenetre 1 --reference normale
uv run python scripts/drift_lab.py --fenetre 2 --reference normale
uv run python scripts/drift_lab.py --fenetre 3 --reference normale
uv run python scripts/drift_lab.py --fenetre janvier --reference normale
uv run python scripts/drift_lab.py --fenetre janvier --reference haute
uv run python scripts/alerting_demo.py --report-out reports/drift_report_f2.json
```

Lire le rapport et rechercher les garde-fous :



```
# Windows
Get-Content -LiteralPath .\reports\drift_report_f2.json
Select-String -LiteralPath .\scripts\alerting_demo.py -Pattern 'drift_events','cooldown_hours','INSERT INTO'
```

```
# macOS et Linux
cat ./reports/drift_report_f2.json
grep -nE 'drift_events|cooldown_hours|INSERT INTO' ./scripts/alerting_demo.py
```

13. M33 et M34 - Prometheus, Grafana et Locust

Si le pack et le dépôt sont deux dossiers différents, résolvez leurs chemins.

```
# Windows, lancé depuis la racine du pack
$packRoot = (Get-Location).Path
$projectRoot = (Resolve-Path -LiteralPath (Read-Host 'Chemin complet du dépôt CISIA')).Path
$payload = Join-Path $packRoot 'payload.json'
$locustfile = Join-Path $packRoot 'perf\locustfile.py'
@($projectRoot, $payload, $locustfile) | ForEach-Object {
    if (-not (Test-Path -LiteralPath $_)) { throw "Introuvable : $_" }
}
Set-Location -LiteralPath $projectRoot
```

```
# macOS ou Linux, lancé depuis la racine du pack
set -euo pipefail
pack_root="$(pwd -P)"
read -r -p 'Chemin complet du dépôt CISIA : ' project_input
project_root="$(cd "$project_input" && pwd -P)"
payload="$pack_root/payload.json"
locustfile="$pack_root/perf/locustfile.py"
for path in "$project_root" "$payload" "$locustfile"; do
    test -e "$path" || { echo "Introuvable : $path" >&2; exit 1; }
done
cd "$project_root"
```

Terminal 1, commande commune à laisser active :

```
uv run python scripts/export_drift_metrics.py
```

Terminal 2 :

```
docker compose config -q
docker compose up -d --build
docker compose ps
```

Tester les métriques :

```
# Windows
Invoke-WebRequest http://127.0.0.1:8000/metrics -UseBasicParsing
Invoke-WebRequest http://127.0.0.1:9109/metrics -UseBasicParsing
```

```
# macOS et Linux
curl -fsS http://127.0.0.1:8000/metrics | head
curl -fsS http://127.0.0.1:9109/metrics | head
```

Interfaces communes : Prometheus <http://127.0.0.1:9090/targets> et Grafana

<http://127.0.0.1:3000>. Locust est lancé headless pendant 30 secondes : aucune interface :8089 n'est ouverte dans ce scénario.

Prometheus tourne dans un conteneur alors que l'exporteur drift tourne sur

l'hôte. Le nom `host.docker.internal` est automatique sur Docker Desktop. Pour

Docker Engine sous Linux, le service `prometheus` doit contenir :

```
extra_hosts:
  - "host.docker.internal:host-gateway"
```

Vérification portable via l'API de Prometheus :

```
# Windows
(Invoke-RestMethod http://127.0.0.1:9090/api/v1/targets).data.activeTargets |
Select-Object -ExpandProperty health
```




```
# macOS et Linux
curl -fsS http://127.0.0.1:9090/api/v1/targets | uv run python -m json.tool
```

Lancer Locust depuis le même terminal où les chemins ont été définis :

```
# Windows
uv run --with locust==2.44.4 locust -f $locustfile --headless -u 20 -r 5 -t 30s --host http://127.0.0.1:8000
```

```
# macOS et Linux
uv run --with locust==2.44.4 locust -f "$locustfile" --headless -u 20 -r 5 -t 30s --host http://127.0.0.1:8000
```

`--with locust==2.44.4` fournit l'outil de façon éphémère sans modifier `pyproject.toml` ni `uv.lock`; `--headless` retire l'UI, `-u 20` fixe les utilisateurs, `-r 5` leur rythme de démarrage et `-t 30s` borne strictement la charge.

14. J6 - Game Day hors ligne

Depuis la racine du parcours :

```
# Windows
powershell -ExecutionPolicy Bypass -File .\scripts\formation\demarrer_gameday.ps1 -Binome equipe-1
```

```
# macOS et Linux
bash scripts/formation/demarrer_gameday.sh equipe-1
```

Le script clone le bundle local, crée `reparation-equipe-1` et vérifie le tag `v1.0-sain`. Il refuse une destination déjà présente. Ne fusionnez jamais la branche cassée dans votre dépôt InduSense habituel.

15. Dépannage par système

Port 8000 déjà occupé

```
# Windows
Get-NetTCPConnection -LocalPort 8000 -ErrorAction SilentlyContinue
```

```
# macOS
lsof -nP -iTCP:8000 -sTCP:LISTEN
```

```
# Linux
ss -ltnp | grep ':8000 '
```

Identifiez d'abord le processus. Ne le terminez que s'il vous appartient et si vous savez à quel exercice il correspond.

Script shell illisible sous macOS ou Linux

Utilisez `bash scripts/...` comme indiqué. Si le message contient `$'\r'` ou `bad interpreter`, le fichier a reçu des fins de ligne Windows : repartez du jalon officiel au lieu de réécrire le script à la main.

Chemin trop long ou synchronisé

- ▶ Windows : utilisez `C:\CISIA\S3`.
- ▶ macOS et Linux : utilisez `~/CISIA/S3`.
- ▶ Évitez OneDrive, iCloud Drive, un partage réseau et les chemins contenant de nombreuses imbrications pour les environnements Python et les volumes Docker.

Différence entre l'hôte et un conteneur

`localhost` désigne la machine qui exécute la commande. Depuis votre navigateur, `127.0.0.1:8000` vise le port publié sur l'hôte. Depuis Prometheus dans Compose,



`api:8000` vise le service `api`, et `host.docker.internal:9109` vise l'exporteur qui tourne sur l'hôte.

16. Ce qui ne change pas selon le système

- ▶ mêmes fichiers source et même `uv.lock` ;
- ▶ mêmes versions Python et dépendances ;
- ▶ mêmes tests et critères de réussite ;
- ▶ mêmes statuts HTTP et mêmes preuves ;
- ▶ mêmes règles Git, sécurité et absence de secrets ;
- ▶ mêmes horaires, pauses et livrables.

Références techniques officielles consultées : documentation d'installation et d'environnements `uv` sur <https://docs.astral.sh/uv/>, documentation Docker

Compose sur <https://docs.docker.com/compose/>.